

Lab Exercise #10

Task 2: Write three Assembly language procedures to

- Clear the screen. with the name CLRSCR,
- Set the cursor at top left corner of the screen, with the name SETCRSR,
- Display the table of any number from 2 to 9, with the name DISPTBL

Call these procedures in a main program.

```

TITLE LAB 10 ACTIVITY 2                ; Title of the program
.MODEL SMALL                          ; Memory model for the program
.STACK 100H                           ; Stack size
.DATA
MSG1 DB 'ENTER ANY SINGLE DIGIT NUMBER TO PRINT TABLE: $'    ; Define a string message

DIG DB ?                               ; Declare a variable to store a single-digit number

.CODE
MAIN PROC
    MOV AX,@DATA                      ; Load the data segment address to AX
    MOV DS,AX                         ; Initialize DS register with the data segment address

    CALL CLRSCR                       ; Call procedure to clear the screen
    CALL SETCRSR                      ; Call procedure to set the cursor at the top-left corner
    CALL DISPTBL                      ; Call procedure to display the table

    MOV AH,4CH                        ; DOS function to exit the program
    INT 21H                           ; DOS interrupt to exit
MAIN ENDP

CLRSCR PROC
    MOV AX,0600H                      ; Function to set video mode
    MOV BH,07                         ; Page number (0 to 7)
    MOV CX,0000                       ; Upper-left corner row
    MOV DX,184FH                      ; Lower-right corner row and column
    INT 10H                           ; Video interrupt to set video mode
    RET                               ; Return from the procedure
CLRSCR ENDP

SETCRSR PROC
    MOV AH,02                         ; Function to set cursor position
    MOV BH,00                         ; Video page number (0 for default)
    MOV DL,00                         ; Column (top-left corner)
    MOV DH,03                         ; Row (top-left corner)
    INT 10H                           ; Video interrupt to set cursor position
    RET                               ; Return from the procedure
SETCRSR ENDP

DISPTBL PROC
    MOV AX,0                          ; Initialize AX to 0
    MOV BX,0                          ; Initialize BX to 0
    MOV CX,0                          ; Initialize CX to 0
    MOV DX,0                          ; Initialize DX to 0

```

```
MOV AH,9                ; Function to print a string
LEA DX,MSG1             ; Load the offset address of the message
INT 21H                 ; DOS interrupt to print the message

MOV AH,1                ; Function to read a character
INT 21H                 ; DOS interrupt to read a character
MOV CX,00001010B        ; Set the limit for the table (e.g., 10 times)
MOV DH,1                ; Set the initial value for the table
AND AL,0FH              ; And AL with 0fh
MOV DIG,AL              ; mov al to dig

AGAIN:
MOV AH,2H               ; Function to move the cursor
MOV DL,0AH              ; Linefeed character
INT 21H                 ; DOS interrupt to move the cursor

MOV DL,0DH              ; Carriage return character
INT 21H                 ; DOS interrupt to move the cursor

MOV AL,DIG              ; Display the current number
OR AL,30H               ; Convert the number to ASCII
MOV AH,2                ; Function to display a character
MOV DL,AL               ; Move the character to DL
INT 21H                 ; DOS interrupt to display a character

MOV DL,'x'              ; Display the multiplication symbol
INT 21H                 ; DOS interrupt to display a character

MOV AL,DH               ; Display the current iteration
AAM                     ; ASCII adjust AL to unpack binary-coded decimal
OR AX,3030H             ; Convert result to ASCII in AX
MOV BX,AX               ; Move the ASCII value to BX

MOV AH,2                ; Function to display a character
MOV DL,BH               ; Move the high digit of the result to DL
INT 21H                 ; DOS interrupt to display a character

MOV DL,BL               ; Move the low digit of the result to DL
INT 21H                 ; DOS interrupt to display a character

MOV DL,'='              ; Display equal sign
INT 21H                 ; DOS interrupt to display a character

MOV DL,20H              ; Display a space
INT 21H                 ; DOS interrupt to display a character

MOV AL,DIG              ; Calculate and display the result
MUL DH                  ; Multiply AL by DH
AAM                     ; ASCII adjust AX to unpack binary-coded decimal
OR AX,3030H             ; Convert result to ASCII in AX
MOV BX,AX               ; Move the ASCII value to BX
```

```

MOV AH,2          ; Function to display a character
MOV DL,BH         ; Move the high digit of the result to DL
INT 21H           ; DOS interrupt to display a character

MOV DL,BL         ; Move the low digit of the result to DL
INT 21H           ; DOS interrupt to display a character

INC DH            ; Increment DH for the next iteration

LOOP AGAIN        ; Continue the loop until CX reaches the limit
RET               ; Return from the procedure
DISPTBL ENDP
END MAIN          ; End of the main program

```

emulator screen (80x25 chars)

```

ENTER ANY SINGLE DIGIT NUMBER TO PRINT TABLE: 4
4x01= 04
4x02= 08
4x03= 12
4x04= 16
4x05= 20
4x06= 24
4x07= 28
4x08= 32
4x09= 36
4x10= 40

```

Task 3: Write three Assembly language procedures to

- Take two 1-digit inputs (from 0 to 9) from user, with the name INPNMBR
- Perform the Add, Subtract, Multiply, and Divide operations, name it OPRATIONS,
- Display the results for all the operations on screen, with the name DISPRSLT. Call these procedures in a main program.

```

TITLE LAB 10 ACTIVITY 3          ; Title of the program
.MODEL SMALL                     ; Memory model
.STACK 100H                      ; Stack size
.DATA
FIRST_NUMBER DB 'ENTER FIRST NUMBER: $' ; Prompt for the first number
SECOND_NUMBER DB 0DH, 0AH, 'ENTER SECOND NUMBER: $' ; Prompt for the second number
ADDITION DB 0DH, 0AH, 'THE RESULT OF ADDITION IS = $' ; Message for addition result
MULTIPLY DB 0DH, 0AH, 'THE RESULT OF MULTIPLICATION IS = $' ; Message for multiplication result
SUBTRACT DB 0DH, 0AH, 'THE RESULT OF SUBTRACTION IS = $' ; Message for subtraction result
REMAINDER DB 0DH, 0AH, 'THE RESULT OF DIVISION, REMAINDER = $' ; Message for division remainder result
QUOTIENT DB 0DH, 0AH, 'THE RESULT OF DIVISION, QUOTIENT = $' ; Message for division quotient result

VAR1 DB ?                       ; Variable for the first input
VAR2 DB ?                       ; Variable for the second input
A1 DB ?                         ; Variables for addition result
A2 DB ?
M1 DB ?                         ; Variables for multiplication result
M2 DB ?

```

```
S1 DB ? ; Variables for subtraction result
S2 DB ?
Q DB ? ; Variables for division quotient result
R DB ? ; Variable for division remainder result
```

```
.CODE
MAIN PROC
    MOV AX,@DATA ; Load the address of the data segment to AX
    MOV DS,AX ; Initialize the data segment register with the address in AX

    CALL INPNMBR ; Call input number procedure
    CALL OPERATIONS ; Call operations procedure
    CALL DISPRSLT ; Call display result procedure

    MOV AH,4CH ; DOS function to terminate program
    INT 21H ; Call DOS interrupt
MAIN ENDP
```

```
INPNMBR PROC
    LEA DX,FIRST_NUMBER ; Load effective address of the first number prompt to DX
    MOV AH,9 ; DOS function to display a string
    INT 21H ; Call DOS interrupt

    MOV AH,1 ; DOS function to read a character from standard input
    INT 21H ; Call DOS interrupt
    MOV VAR1,AL ; Move the input to VAR1

    LEA DX,SECOND_NUMBER ; Load effective address of the second number prompt to DX
    MOV AH,9 ; DOS function to display a string
    INT 21H ; Call DOS interrupt

    MOV AH,1 ; DOS function to read a character from standard input
    INT 21H ; Call DOS interrupt
    MOV VAR2,AL ; Move the input to VAR2

    RET ; Return from the procedure
INPNMBR ENDP
```

```
OPERATIONS PROC
;ADDITION:
    MOV AX,00 ; Initialize AX to 0
    MOV BX,00 ; Initialize BX to 0

    MOV BL,VAR1 ; Move VAR1 to BL
    MOV AL,VAR2 ; Move VAR2 to AL
    ADD AL,BL ; Add VAR2 to VAR1
    AAA ; ASCII adjust AL after addition
    OR AX,3030H ; Convert result to ASCII in AX
    MOV BX,AX ; Move the ASCII value to BX
    MOV A1,BH ; Store high digit of result in A1
    MOV A2,BL ; Store low digit of result in A2
```

;SUBTRACTION:

```
MOV AX,00          ; Initialize AX to 0
MOV BX,00          ; Initialize BX to 0

MOV BL,VAR2        ; Move VAR2 to BL
MOV AL,VAR1        ; Move VAR1 to A
AND AL,0FH
AND BL,0FH
SUB AL,BL          ; Subtract VAR2 from VAR1

AAS                ; ASCII adjust AL after subtraction
OR AX,3030H        ; Convert result to ASCII in AX
MOV BX,AX          ; Move the ASCII value to BX
MOV S1,BH          ; Store high digit of result in S1
MOV S2,BL          ; Store low digit of result in S2
```

;MULTIPLICATION:

```
MOV AX,00          ; Initialize AX to 0
MOV BX,00          ; Initialize BX to 0

MOV BL,VAR1        ; Move VAR1 to BL
MOV AL,VAR2        ; Move VAR2 to AL
AND AL,0FH
AND BL,0FH

MUL BL             ; Multiply VAR1 by VAR2
AAM               ; ASCII adjust AX to unpack binary-coded decimal
OR AX,3030H       ; Convert result to ASCII in AX
MOV BX,AX         ; Move the ASCII value to BX
MOV M1,BH         ; Store high digit of result in M1
MOV M2,BL         ; Store low digit of result in M2
```

;DIVISION:

```
MOV AX,00          ; Initialize AX to 0
MOV BX,00          ; Initialize BX to 0

MOV BL,VAR1        ; Move VAR1 to BL
MOV AL,VAR2        ; Move VAR2 to AL
XCHG AL,BL         ; Exchange AL and BL registers
AAD               ; ASCII adjust AX to unpack binary-coded decimal
DIV BL             ; Divide AX by BL; quotient in AL, remainder in AH
OR AX,3030H       ; Convert quotient to ASCII in AX
MOV R,AH           ; Move remainder to R
MOV Q,AL           ; Move quotient to Q
```

```
RET                ; Return from the procedure
```

OPERATIONS ENDP

DISPRSLT PROC

;ADDITION:

```
LEA DX,ADDITION    ; Load effective address of the result message to DX
MOV AH,9           ; DOS function to display a string
INT 21H            ; Call DOS interrupt
```

```
MOV AH,2           ; DOS function to display a character
MOV DL,A1          ; Move the high digit of the result to DL
INT 21H            ; Call DOS interrupt
MOV AH,2           ; DOS function to display a character
MOV DL,A2          ; Move the low digit of the result to DL
INT 21H            ; Call DOS interrupt
```

;SUBTRACTION:

```
LEA DX,SUBTRACT    ; Load effective address of the result message to DX
MOV AH,9            ; DOS function to display a string
INT 21H            ; Call DOS interrupt
MOV AH,2            ; DOS function to display a character
MOV DL,S1           ; Move the high digit of the result to DL
INT 21H            ; Call DOS interrupt
MOV AH,2            ; DOS function to display a character
MOV DL,S2           ; Move the low digit of the result to DL
INT 21H            ; Call DOS interrupt
```

;MULTIPLICATION:

```
LEA DX,MULTIPLY     ; Load effective address of the result message to DX
MOV AH,9            ; DOS function to display a string
INT 21H            ; Call DOS interrupt
MOV AH,2            ; DOS function to display a character
MOV DL,M1           ; Move the high digit of the result to DL
INT 21H            ; Call DOS interrupt
MOV AH,2            ; DOS function to display a character
MOV DL,M2           ; Move the low digit of the result to DL
INT 21H            ; Call DOS interrupt
```

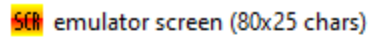
;DIVISION:

```
LEA DX,QUOTIENT     ; Load effective address of the result message to DX
MOV AH,9            ; DOS function to display a string
INT 21H            ; Call DOS interrupt
MOV AH,2            ; DOS function to display a character
MOV DL,Q            ; Move the high digit of the result to DL
INT 21H            ; Call DOS interrupt
```

```
LEA DX,REMAINDER    ; Load effective address of the result message to DX
MOV AH,9            ; DOS function to display a string
INT 21H            ; Call DOS interrupt
MOV AH,2            ; DOS function to display a character
MOV DL,R            ; Move the high digit of the result to DL
INT 21H            ; Call DOS interrupt
```

```
RET                 ; Return from the procedure
DISPRSLT ENDP
```

```
END MAIN           ; End of the program
```

 emulator screen (80x25 chars)

```
ENTER FIRST NUMBER: 4
ENTER SECOND NUMBER: 4
THE RESULT OF ADDITION IS = 08
THE RESULT OF SUBTRACTION IS = 00
THE RESULT OF MULTIPLICATION IS = 16
THE RESULT OF DIVISION, QUOTIENT = 1
THE RESULT OF DIVISION, REMAINDER = 0
```

Lab Exercise #11

Task 1: Write a program using a look-up table and XLAT to retrieve the value of y in the following equation:

$$y = x^2 + 10$$

where, x value ranges from 0 to 9.

```

TITLE LAB 11 ACTIVITY 3                ; Title directive

.MODEL SMALL                          ; Memory model for the program
.STACK 100H                           ; Stack size

.DATA                                ; Data segment
VAL db 10,11,14,19,26,35,46,59,74,91 ; Array of values

MSG DB 'ENTER THE VALUE OF X(0-9) TO GET VALUE OF Y=(X^2)+10 : $' ; Prompt message
MSG1 DB 'VALUE OF Y FOR ENTERED VALUE OF X = $' ; Result message

.CODE                                ; Code segment
MAIN PROC
    MOV AX,@DATA                    ; Load data segment address to AX
    MOV DS,AX                      ; Initialize DS register with the data segment address

    MOV AH,9                       ; DOS function to print a string
    LEA DX,MSG                     ; Load the offset address of the prompt message
    INT 21H                       ; DOS interrupt to print the message

    mov ah,1                      ; DOS function to read a character from standard input
    int 21h                      ; DOS interrupt
    AND AL,0FH                   ; Mask high bits to get the numeric value
    MOV CL,AL                    ; Store the entered value in CL

    MOV AH,2                      ; DOS function to move the cursor
    MOV DL,0AH                   ; Linefeed character
    INT 21H                      ; DOS interrupt

    MOV AH,2                      ; DOS function to move the cursor
    MOV DL,0DH                   ; Carriage return character
    INT 21H                      ; DOS interrupt

    MOV AH,9                     ; DOS function to print a string
    MOV DX,OFFSET MSG1           ; Load the offset address of the result message
    INT 21H                     ; DOS interrupt

    LEA BX,VAL                   ; Load the offset address of the array VAL to BX
    MOV AL,CL                   ; Move the entered value to AL
    XLAT                        ; Translate AL using the contents of the array VAL

    AAM                         ; ASCII adjust AX to unpack binary-coded decimal
    OR AX,3030H                 ; Convert result to ASCII in AX

    MOV BX,AX                   ; Move the ASCII value to BX

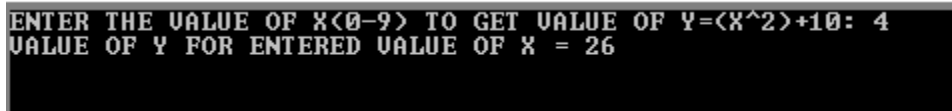
```

```
MOV AH,2                ; DOS function to display a character
MOV DL,BH                ; Move the high digit of the result to DL
INT 21H                  ; DOS interrupt

MOV AH,2                ; DOS function to display a character
MOV DL,BL                ; Move the low digit of the result to DL
INT 21H                  ; DOS interrupt

MOV AH,4CH               ; DOS function to exit the program
INT 21H                  ; DOS interrupt

MAIN ENDP
END MAIN                 ; End of the program
```



```
ENTER THE VALUE OF X<0-9> TO GET VALUE OF Y=<X^2>+10: 4
VALUE OF Y FOR ENTERED VALUE OF X = 26
```